# Developer Manual

Group 44

This guide explains how to configure, build, run, debug and test the command-line Personal Information Manager (PIM). Start in the `PIM` project folder, which contains `src/`, `scripts/` and `lib/`.

## 1 Development environment

Download and extract the project to your local machine running macOS. Make sure Java Development Kit (JDK) version 17 or later has been installed. Open the project in JetBrains IntelliJ IDEA version 2025.2.1 or later. The reference environment uses OpenJDK 24.0.2 and IntelliJ IDEA 2025.2.1. Compilation targets Java 17; set the IDE language level to 17. Terminal scripts require Bash and the JDK tools `java`, `javac` and `jar` on `PATH`.

The application uses the Java standard library. Test dependencies are bundled in `lib/test/`: `junit-4.13.1.jar` and `hamcrest-core-1.3.jar`. Add them with Test scope in the IDE. The scripts use these local JARs for offline builds and tests.

## 2 Configure the project in IntelliJ IDEA

1. Open the `PIM` folder as the project. In *File > Project Structure > Project*, select an installed JDK 17 or newer and set *Language level* to 17. If the saved SDK name is unavailable, select your installed JDK.
2. In *Modules > Sources*, verify that the content root is the `PIM` folder. Mark `src/main/java` as *Sources* and `src/test/java` as *Test Sources*. Exclude generated `build/` and `out/` folders from source indexing.
3. In *Modules > Dependencies*, use the project SDK. Add both JARs from `PIM/lib/test/` through *JARs or directories* if needed, and set their scope to *Test*. Check that the paths resolve to existing files.
4. In *Project > Compiler output*, verify an output directory such as `PIM/out`. Let the module inherit that output location. Apply the settings before building.

## 3 Build and run in IntelliJ IDEA

1. Choose *Build > Build Project* and check the Build output for errors. With compiler output set to `out`, application classes are placed under `out/production/PIM` for the `PIM` module.
2. Open `src/main/java/app/Main.java` and run `main` using the gutter icon. For a saved Application configuration, use main class `app.Main`, module `PIM`, your selected JDK, and the `PIM` root as the working directory.
3. The Run console should display the welcome message and the `pim>` prompt. Enter `help`, then try `create note text="Check the setup"` and `print all`. Use `exit` to close the session; confirm if changes are unsaved.

## 4 Debug a command

1. Set a breakpoint in `controller.PimController.execute`. Start `app.Main` with *Debug*.
2. Enter a command in the Debug console. When execution pauses, inspect the parsed `command` and step into the selected model call. For an update, inspect the validated replacement before it is stored.
3. Use *Step Over*, *Step Into* and *Resume Program* to follow execution and return to the prompt.

# 5 Build and run from a terminal

Run these commands from the `PIM` source root:

```
java -version
javac -version
./scripts/build.sh
java -jar build/pim.jar
```

The build script compiles with `-release 17`, UTF-8 encoding, debug information and `-Xlint:all -Werror`. It writes application classes to `build/classes/` and creates `build/pim.jar` with `app.Main` as its entry point. This script produces the executable JAR used for command-line distribution. Relative save/load paths are resolved from the application's working directory.

# 6 Run tests and inspect coverage

```
./scripts/test.sh
```

The script rebuilds the application, compiles tests into `build/test-classes/` and runs JUnit 4.13.1. The current suite reports `OK (28 tests)` for `RecordTests`, `CriteriaTests` and `StorageTests`, followed by `OK (7 tests)` for `ConsoleTests`.

In IntelliJ, run a test class or method using its gutter icon. For model coverage, select the shared *PIM Model Coverage* JUnit configuration and choose *Run with Coverage*. It runs the three model test classes, uses JaCoCo and restricts coverage to `model.*`. To inspect controller and view coverage as well, run all tests in the `tests` package with coverage and adjust the coverage filter to include those application packages. The Model Line Coverage report contains the recorded results and screenshots.

# 7 Source responsibilities

| Package / directory | Responsibility |
|---|---|
| `app` | Constructs services and starts the session. |
| `controller`, `view` | Parses commands, tracks session state and handles terminal input/output. |
| `model` | Manages the active record collection, IDs and snapshots. |
| `model.records` | Defines PIR types and validates their fields. |
| `model.criteria` | Tokenizes, parses and evaluates search expressions. |
| `model.persistence`, `model.exceptions` | Reads/writes `.pim` snapshots and defines model errors. |
| `src/test/java/tests` | Contains model unit tests and console scenarios. |

# 8 Resolve setup problems

- **Missing JDK tools or unsupported release.** Check `java -version` and `javac -version`; select a JDK 17 or newer in both the terminal environment and the IDE.
- **Missing main class, JUnit imports or test icons.** Recheck the content root, both source roots, module classpath and Test-scope JARs. Rebuild after correcting them.
- **Script permission or file-path errors.** If executable permissions were lost during extraction, run `chmod u+x scripts/build.sh scripts/test.sh` before retrying. Verify that the working directory is `PIM` and that any save destination directory exists.